

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

Satellite based disaster monitoring and alert system:

AI-Powered Satellite Image Analysis and Real-Time Disaster Alert System



Prepared by:

BATHAKARI VISHNU

J. ADILAKSHMI

PODRU VENKATA DEVI

MADINENI VENKATA ROHITH

Batch No: 132

Selected Batch:

Batch 1

Mentor: Mrs Bethala Sumana

Organisation:

Andhra Pradesh State Skill Development Corporation (APSSDC)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the Andhra Pradesh Skill Development Corporation (APSSDC) for providing me with the opportunity to participate in this internship program. This internship has given me valuable practical knowledge and hands-on experience in cloud computing and modern DevOps practices using Amazon Web Services (AWS).

I am especially thankful to our mentor, Mrs. Sumana Bethala, for her continuous guidance, encouragement, and valuable suggestions throughout the internship. Her support and motivation helped us understand the concepts clearly and successfully complete our project.

I would also like to thank our project team members for their cooperation, coordination, and dedication. Working together as a team helped us successfully develop different modules of the project and gain practical experience in collaborative software development.

I extend my heartfelt thanks to the faculty members of our college for their support and encouragement throughout the internship period.

Finally, I express my gratitude to everyone who directly or indirectly contributed to the successful completion of our project, "Satellite-Based Disaster Monitoring and Alert System." This internship has enhanced my technical knowledge, problem-solving abilities, teamwork, and understanding of real-world cloud technologies, which will be valuable for my future career.

Thank you

Sincerely,

BATHAKARI VISHNU

J. ADILAKSHMI

PODRU VENKATA DEVI

MADINENI VENKATA ROHITH

ABSTRACT

The Satellite-Based Disaster Monitoring and Alert System is a cloud-based application designed to monitor and manage disaster-related information using satellite images and AWS cloud services. The system helps in collecting, processing, storing, monitoring, and delivering disaster-related information efficiently through a scalable and reliable cloud infrastructure.

In this project, satellite images and disaster-related data are uploaded to Amazon S3. The uploaded data triggers AWS Lambda, which processes the information and stores the required disaster details in Amazon DynamoDB. Amazon API Gateway provides secure REST API access for communication between the application and backend services. Amazon CloudWatch is used for monitoring and logging system activities, while Amazon SNS is used to send disaster alerts and notifications. AWS IAM provides secure access control and permissions for the AWS services used in the system.

The system follows a serverless and event-driven architecture, which improves automation, scalability, reliability, and security. By integrating multiple AWS services, the proposed system provides an efficient platform for disaster monitoring and timely alert generation. This project demonstrates the practical application of cloud computing, serverless technologies, API integration, storage services, database management, monitoring, notification services, and identity and access management in developing a real-world disaster management solution.

Keywords: Satellite-Based Disaster Monitoring, Disaster Management, Cloud Computing, Amazon S3, AWS Lambda, Amazon DynamoDB, Amazon SNS, Amazon CloudWatch, Amazon API Gateway, AWS IAM, Serverless Architecture.

Services mentioned: S3, Lambda, DynamoDB, SNS, CloudWatch, API Gateway, and IAM.

Table of Contents

- 1.Introduction
 - 1.1 Purpose
 - 1.2 Scope
- 2. Description
 - 2.1 Existing System
 - 2.2 Proposed System
 - 2.3 Product Features
 - 2.4 User Classes
- 3. Functional Requirements
- 4. Non-Functional Requirements
- 5. Tech Stack
- 6. System Architecture
- 7. Explanation
 - 7.1 Storage Layer
 - 7.2 Aws services used
- 9 Project output
- 10. Project Links
- 11. Conclusion
- 12. References

1. Introduction

The Satellite-Based Disaster Monitoring System

The **Satellite-Based Disaster Monitoring System** is a web-based application developed to detect, classify, and monitor natural disasters using satellite and aerial imagery. The system allows users to upload satellite images, analyzes them using an AI-powered ensemble model, and classifies disasters into categories such as **Flood, Fire, Cyclone**, and **Normal**.

Based on the classification result, the system assigns a **confidence score** and **severity level** (High / Medium / Low).

High-severity disasters trigger **real-time alerts** via AWS SNS.

The platform provides an interactive **dashboard, map view, reports**, and **admin management** to support disaster response teams and administrators in monitoring affected regions across India.

1.1 Purpose

The purpose of the Satellite-Based Disaster Monitoring System is to provide a smart and efficient platform for **early disaster detection and monitoring** using Machine Learning and Deep Learning.

The system analyzes uploaded satellite images to:

- Classify disaster type (Flood, Fire, Cyclone, Normal)
- Calculate confidence and severity
- Store disaster records with location coordinates
- Send alerts for high-severity events
- Display analytics on an interactive dashboard and map

The platform helps disaster management teams, administrators, and users make **informed decisions** during emergencies and improves response time through automated AI analysis.

1.2 Scope

The system is designed for disaster monitoring and emergency response teams. It allows users to register, log in, upload satellite images, receive AI-based disaster classifications, view reports on an interactive map, and manage disaster lifecycle status.

The platform securely stores disaster records, supports AWS cloud integration (S3, DynamoDB, SNS, Lambda), and provides a responsive web interface for real-time monitoring and administration.

2. Description

The Satellite-Based Disaster Monitoring System is a web-based application that assists users in detecting and tracking natural disasters from satellite imagery. Users upload images along with a location (e.g., Kerala, Andhra Pradesh, Odisha Coast), and the system processes the image through an AI classification pipeline.

Using a **multi-tier AI ensemble** (CLIP k-NN + zero-shot CLIP + fine-tuned MobileNetV2),

the application generates disaster type, confidence percentage, and severity.

High-severity results trigger **SNS email alerts** to notify administrators.

Records are displayed on a **Leaflet-based interactive map** with color-coded markers by disaster type.

Administrators can update lifecycle status (**Active / Resolved**), delete records, and export reports as CSV.

The application is developed using **React + Vite** for the frontend, **FastAPI (Python)** for the backend API, **PyTorch / Transformers (CLIP)** for AI classification, and **AWS** (S3, DynamoDB, SNS, Lambda) for cloud storage and alerting.

2.1 Existing System

Existing disaster monitoring systems often rely on **manual image inspection**, static reporting, or government portals without real-time AI classification. Most platforms do not provide:

- Automated satellite image disaster detection
- Multi-model AI ensemble with graceful fallback
- Interactive map-based visualization
- Real-time high-severity alerting
- Integrated admin lifecycle management

As a result, response teams face delays in identifying disaster type, location impact, and severity during critical events.

2.2 Proposed System

The proposed **Satellite-Based Disaster Monitoring System** uses Machine Learning and Deep Learning to provide automated disaster detection from satellite images.

The system:

- Accepts image uploads with location metadata
- Classifies disasters using a **3-tier AI pipeline** (CLIP ensemble → MobileNet → color heuristic fallback)
- Stores records with coordinates for map display
- Sends **SNS alerts** for high-severity disasters
- Provides dashboard analytics, reports, alerts page, and admin CRUD operations
- Supports both **local development mode** and **AWS cloud deployment**

2.3 Product Features

The major features of the system include:

- Secure user registration, login, and admin login (demo authentication via localStorage)
- Satellite image upload with location selection (India-focused presets)
- AI-powered disaster classification (Flood / Fire / Cyclone / Normal)
- Confidence score and severity assignment (High / Medium / Low)
- Interactive dashboard with statistics (total disasters, by type, by severity)
- Interactive map view with Leaflet markers by disaster type and location
- Disaster detail page with image, coordinates, confidence, and lifecycle
- High-severity alerts page and SNS email notifications
- Admin panel for lifecycle updates (Active / Resolved) and record deletion
- CSV export of disaster reports
- Graceful AI fallback when CLIP model is unavailable (offline MobileNet / heuristic mode)
- Health check API showing AI backend status and system configuration

2.4 User Classes

The system has the following users:

User

- Registers and logs in
- Uploads satellite images with location
- Views dashboard, map, reports, and alerts
- Views individual disaster detail records

Administrator

- Logs in via separate admin portal
- Manages disaster records (update lifecycle, delete)
- Monitors high-severity alerts
- Views system statistics and all reports

Guest (Demo)

- Can access the system in guest mode for demonstration without registration

3. Functional Requirements

Requirement ID	Description
FR-01	The system shall allow users to register and log in with email and password.
FR-02	The system shall allow administrators to log in via a separate admin login page.
FR-03	The system shall allow users to upload satellite images (JPG, PNG, WEBP, BMP; max 15 MB) with a location.
FR-04	The system shall classify uploaded images into Flood, Fire, Cyclone, or Normal using an AI ensemble model.
FR-05	The system shall assign a confidence score (%) and severity level (High / Medium / Low) to each classification.
FR-06	The system shall store disaster records with type, location, coordinates, timestamp, image URL, and lifecycle status.

Requirement ID	Description
FR-07 High	The system shall send SNS email alerts when a disaster is classified as severity.
FR-08 type	The system shall display dashboard statistics (total disasters, breakdown by and severity).
FR-09 with	The system shall display disaster locations on an interactive Leaflet map type-based markers.
FR-10	The system shall allow users to view detailed disaster reports including image and coordinates.
FR-11	The administrator shall update disaster lifecycle status (Active / Resolved).
FR-12	The administrator shall delete disaster records.
FR-13	The system shall export disaster reports as CSV.

- FR-14** The system shall provide a health check endpoint reporting AI backend status and service configuration.
- FR-15** The system shall support sample data seeding and clearing for demonstration purposes.
-

4. Non-Functional Requirements

Performance: Image classification should complete within a few seconds (up to ~2 minutes

on first upload while CLIP model loads/downloads).

Security: User authentication is managed on the client; AWS credentials and SNS topics are

configured for cloud alerting. CORS is restricted to localhost and LAN IP ranges.

Scalability: The application supports multiple concurrent users via FastAPI async endpoints and can scale to AWS (S3, DynamoDB, Lambda, SNS).

Reliability: The AI pipeline degrades gracefully — CLIP ensemble → MobileNet → color heuristic — ensuring the system remains functional even when the CLIP model is unavailable.

Usability: The interface is responsive with sidebar navigation, theme support, and clear status banners when the backend is unreachable.

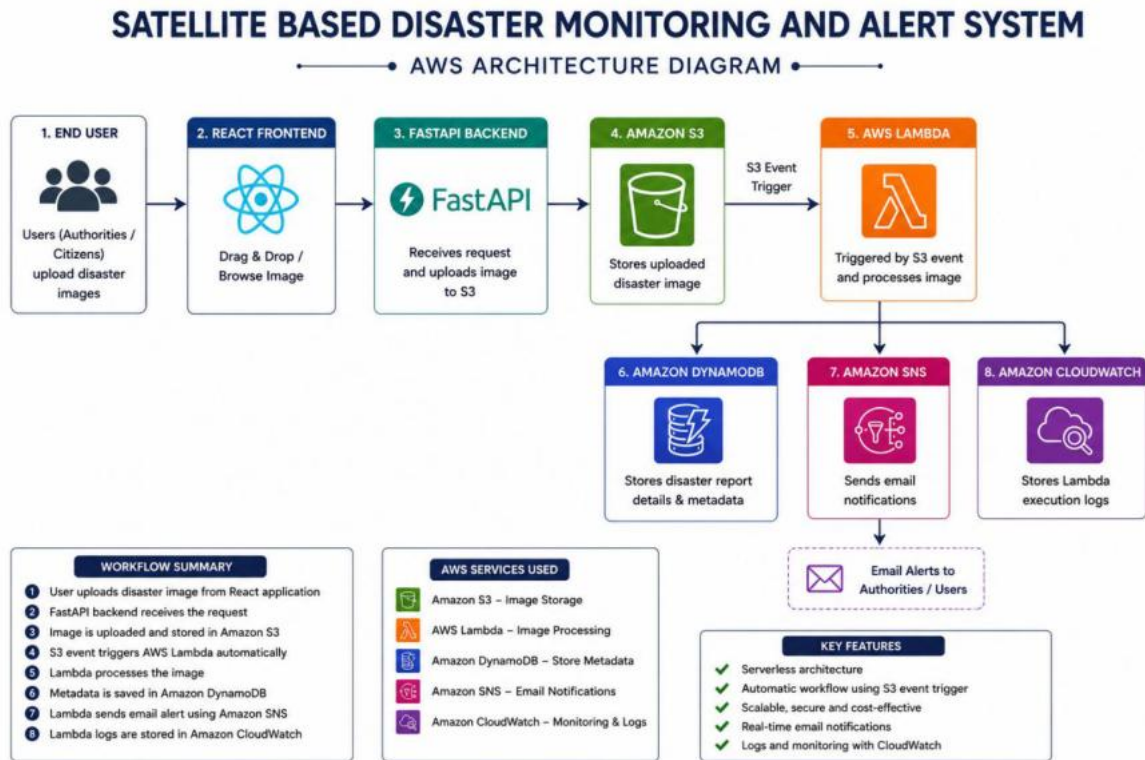
Maintainability: Modular architecture with separate frontend pages, backend services

(classifier, store, alerts, locations), and independent API client.

5. Tech Stack

Layer	Technology
Frontend	React 18, Vite 5, React Router 6
Maps	Leaflet
Backend	FastAPI, Uvicorn
AI / ML	PyTorch, TorchVision, Transformers (CLIP), Scikit-learn, MobileNetV2
Image Processing	Pillow, NumPy
Cloud (AWS)	S3, DynamoDB, SNS, Lambda, API Gateway
Storage (Local Dev)	JSON file (disasters.json), local filesystem uploads
Authentication	Client-side demo auth (localStorage)
Dev Tools	Python 3.11, Node.js, npm

6. System Architecture



Data flow (upload):

1. User uploads image + location via React frontend
 2. FastAPI receives file, saves to local uploads / S3
 3. Classifier runs AI ensemble → returns type, confidence, severity
 4. Record saved to JSON store / DynamoDB with lat/lng from location presets
 5. If High severity → SNS alert published
 6. Frontend refreshes dashboard, map, and alerts
-

7. Explanation

The **Satellite-Based Disaster Monitoring System** is a web-based application developed to assist disaster management teams in detecting and tracking natural disasters from satellite imagery.

The system collects satellite/aerial images through an upload form along with a location preset (Andhra Pradesh, Kerala, Odisha Coast, Tamil Nadu, Assam).

The uploaded image is processed through a **3-tier AI classification pipeline**:

1. **CLIP Ensemble** (k-NN index + zero-shot prompts) — highest accuracy
2. **Fine-tuned MobileNetV2** (disaster_model.pt) — offline fallback
3. **Color Heuristic** — always-available last resort

The model outputs **disaster type**, **confidence (%)**, and **severity** (High / Medium / Low).

High-severity disasters trigger an **AWS SNS notification** with disaster type, location, and confidence details.

The application provides:

- o Secure login (user / admin / guest)
- o Interactive dashboard with live statistics
- o Leaflet map with geo-located disaster markers
- o Detailed report view per disaster record
- o Admin CRUD for lifecycle management
- o CSV export for reporting

The overall objective is to help disaster response teams **detect disasters faster**, **visualize impact geographically**, and **alert stakeholders in real time**.

7.1 Storage Layer

The application uses a **hybrid storage architecture**:

Component	Storage	Purpose
Disaster records	backend/data/disasters.json (local) / DynamoDB (cloud)	Disaster metadata, coordinates, lifecycle
Uploaded images	backend/uploads/ (local) / S3 bucket (cloud)	Satellite image files
AI models	backend/models/ (disaster_model.pt, clip_index.pt) index	Trained MobileNet + CLIP
User sessions	Browser localStorage	Demo authentication state

AWS Cloud Configuration:

1. **S3 Bucket:** 1amazonbucket1332
2. **DynamoDB Table:** disateralert
3. **SNS Topic:** disasteralerts
4. **Lambda Function:** Imageprocess
5. **Region:** eu-north-1
6. **Iam user:** disater-monitoring-alert-2006
7. **Api gateway:** disasteruploadapi

This architecture provides secure, scalable, and real-time disaster data management with optional cloud deployment.

7.2 AWS Services Used

1. Amazon S3



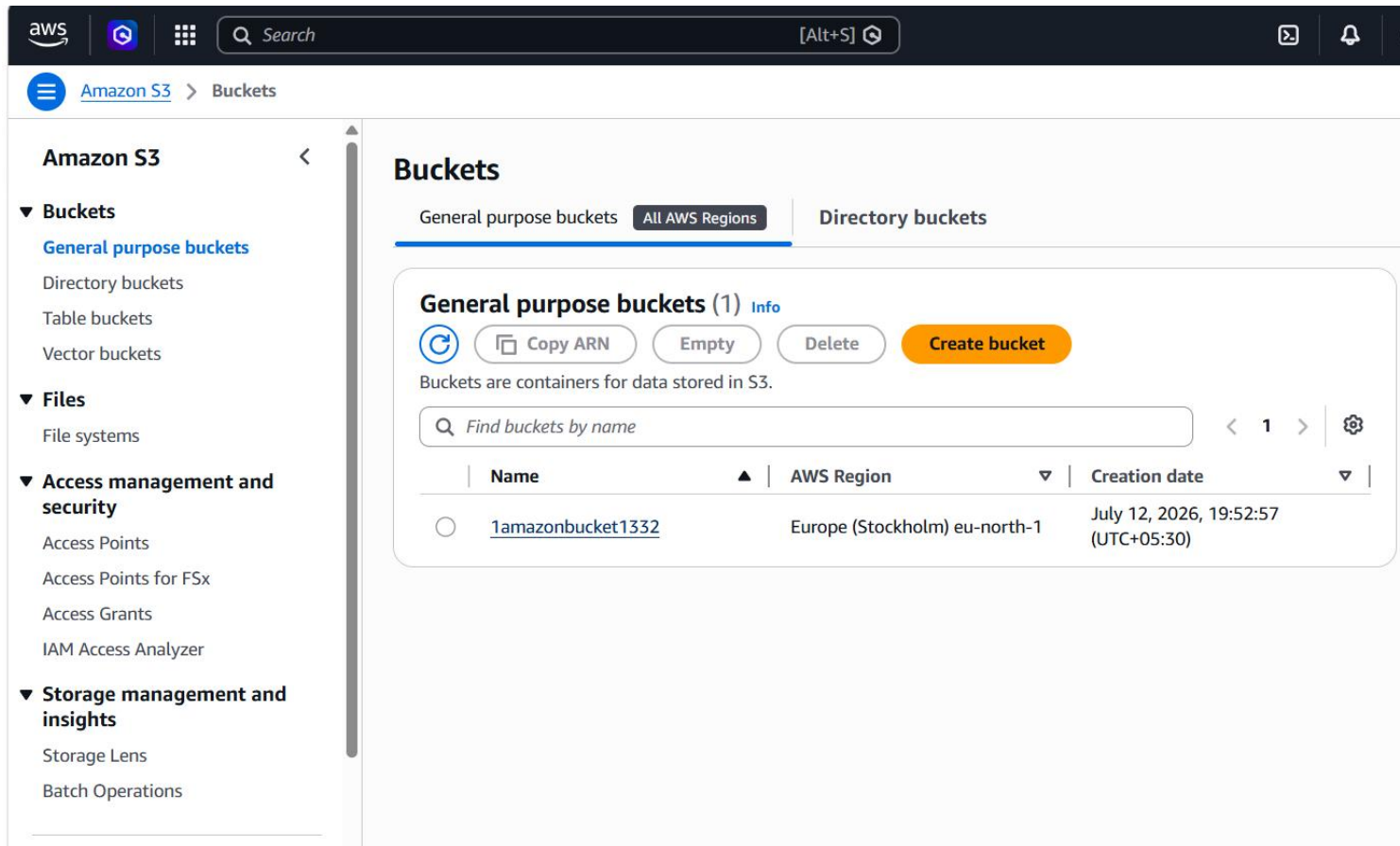
Definition: Amazon S3 (Simple Storage Service) is a cloud object storage service used to securely store and retrieve files.

Usage in Our Project:

Stores uploaded satellite images.

Provides secure and scalable storage.

Triggers AWS Lambda whenever a new image is uploaded.



2. AWS Lambda



Definition: AWS Lambda is a serverless compute service that automatically executes code in response to events.

Usage in Our Project:

Triggered automatically by Amazon S3.

Processes uploaded images.

Stores disaster details in DynamoDB.

Sends notifications using Amazon SNS.

aws

Search

[Alt+S]

Lambda

Functions

Lambda

Dashboard

Applications

Functions

Function resources

Capacity providers

Code signing configurations

Event source mappings

Layers

Replicas

Documentation

Serverless workshops

Related AWS resources

Step Functions state machines

Search functions by name

Functions (1)

Last fetched 15/7/2026, 1:56:03 pm

Actions

Create function

Search by attributes or search by keyword

Function name

Description

Package type

Runtime

Type

Last modified

Imageprocess

-

Zip

Python 3.13

Standard

3 days ago

3. Amazon DynamoDB

The image shows the Amazon DynamoDB logo, which consists of a white lightning bolt striking a stack of four white cylinders, all set against a solid blue square background.

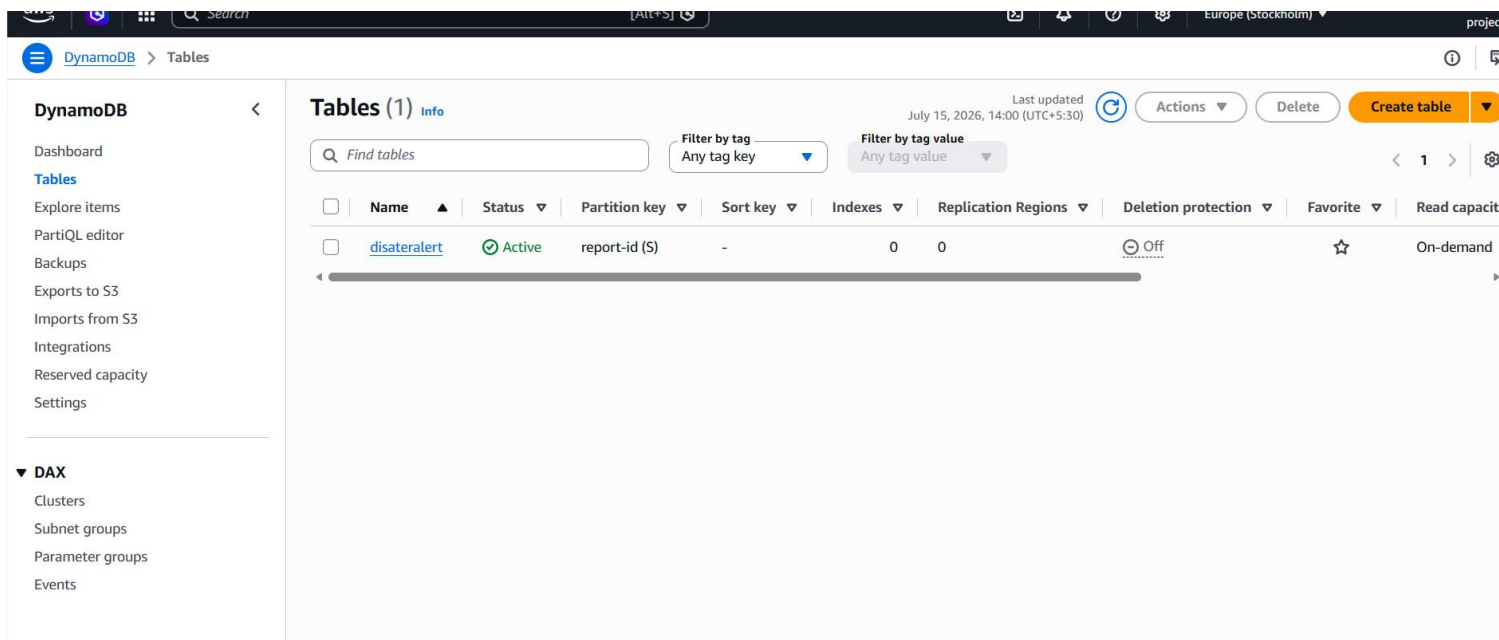
Definition: Amazon DynamoDB is a fully managed NoSQL database service used for storing application data.

Usage in Our Project:

Stores disaster records.

Saves disaster type, location, confidence, severity, and timestamp.

Enables fast retrieval of disaster information.



4. Amazon SNS



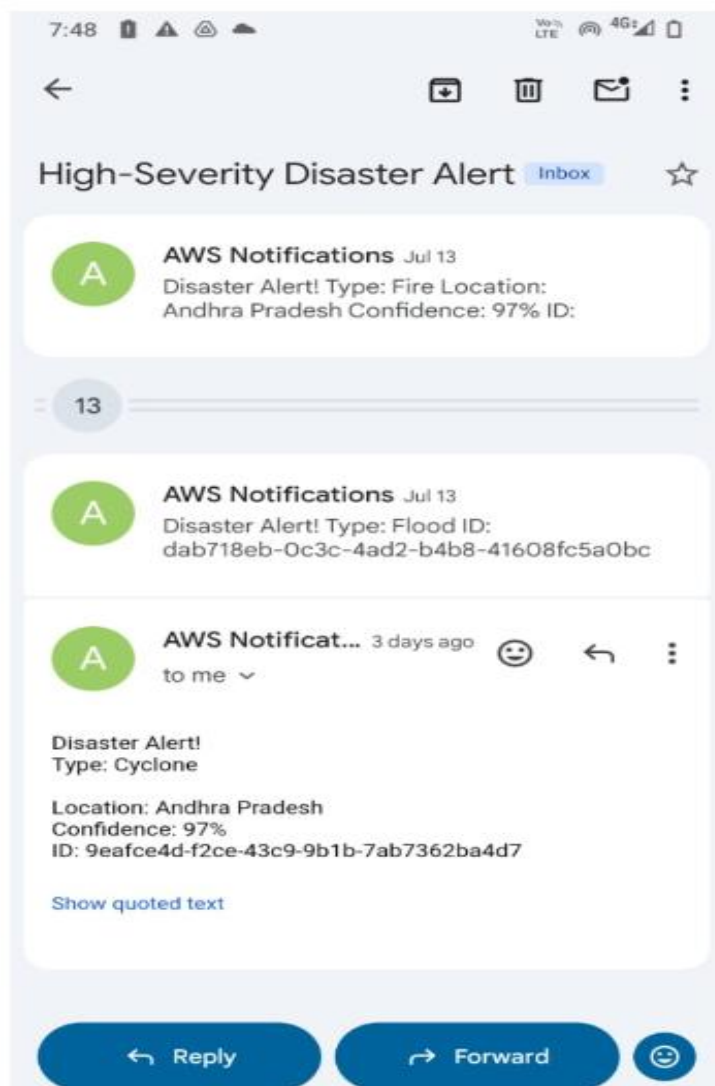
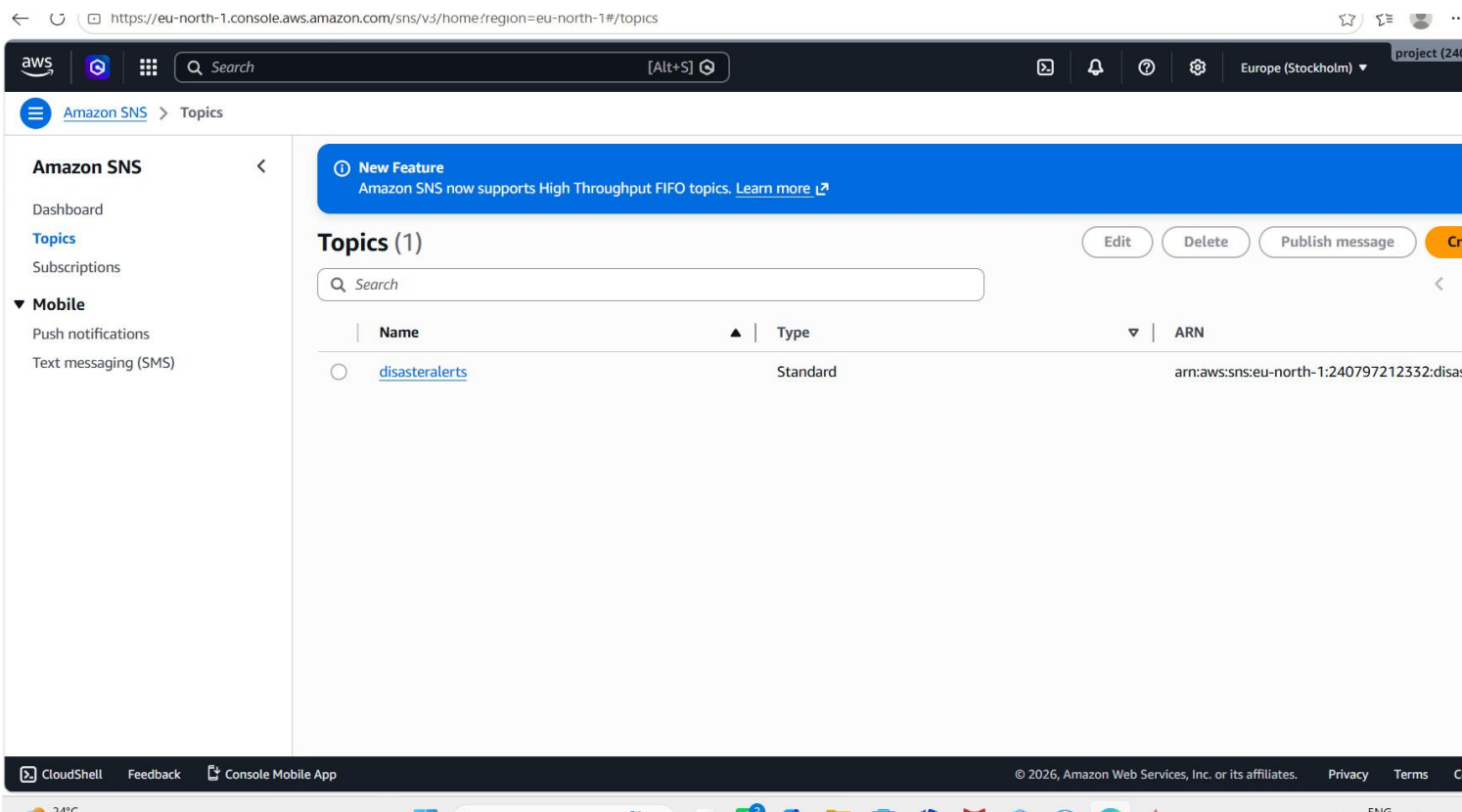
Definition: Amazon SNS (Simple Notification Service) is a messaging service used to send notifications.

Usage in Our Project:

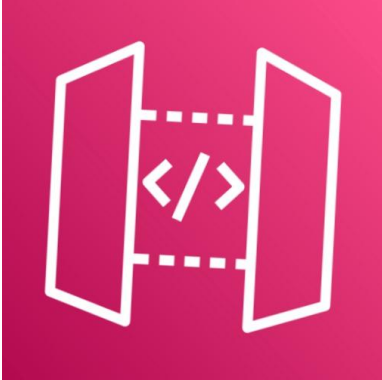
Sends email alerts for high-severity disasters.

Notifies administrators immediately.

Improves emergency response.



5. Amazon API Gateway



Definition: Amazon API Gateway is a managed service used to create and manage REST APIs.

Usage in Our Project:

Connects the frontend with the backend.

Handles API requests securely.

Enables communication between users and AWS services.

6. Amazon CloudWatch



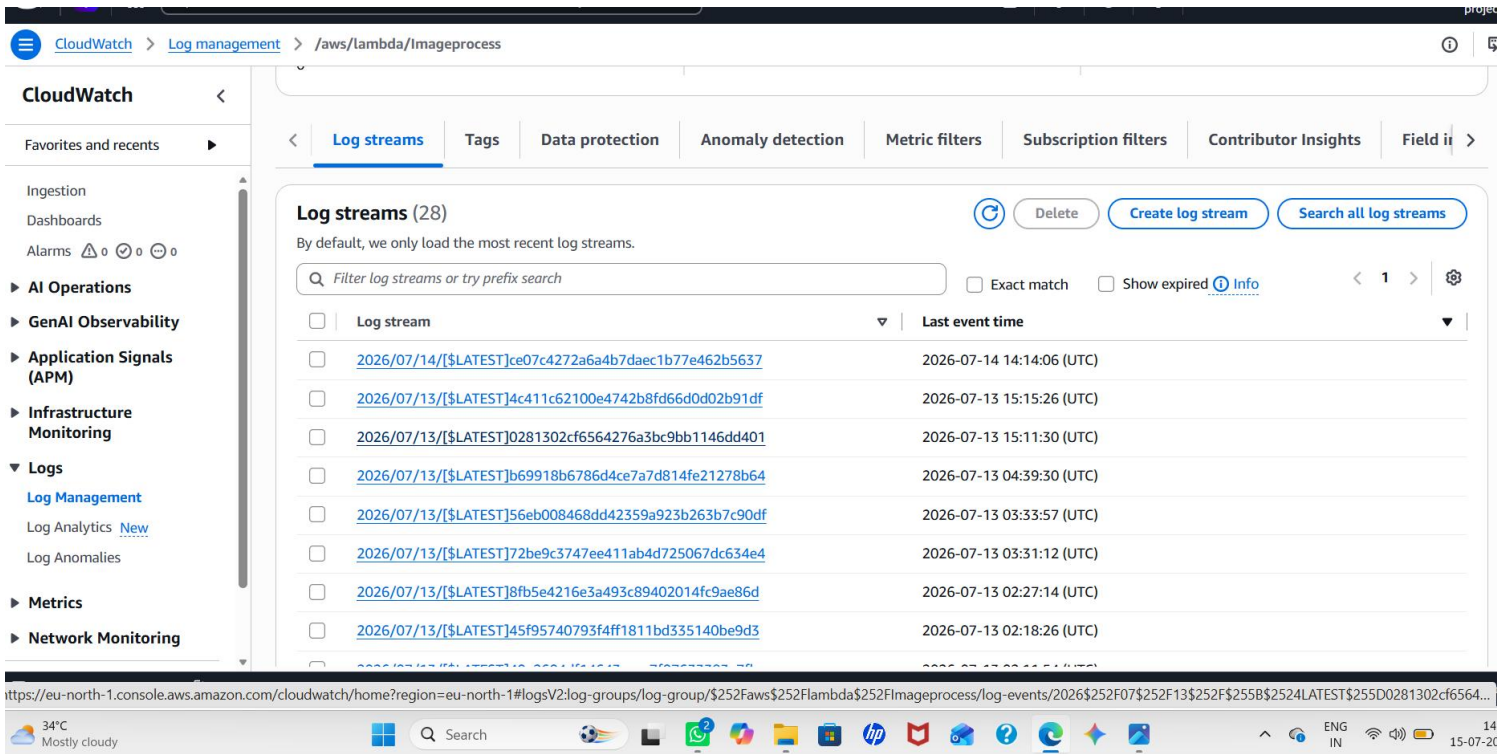
Definition: Amazon CloudWatch is a monitoring and logging service for AWS resources.

Usage in Our Project:

Monitors Lambda execution.

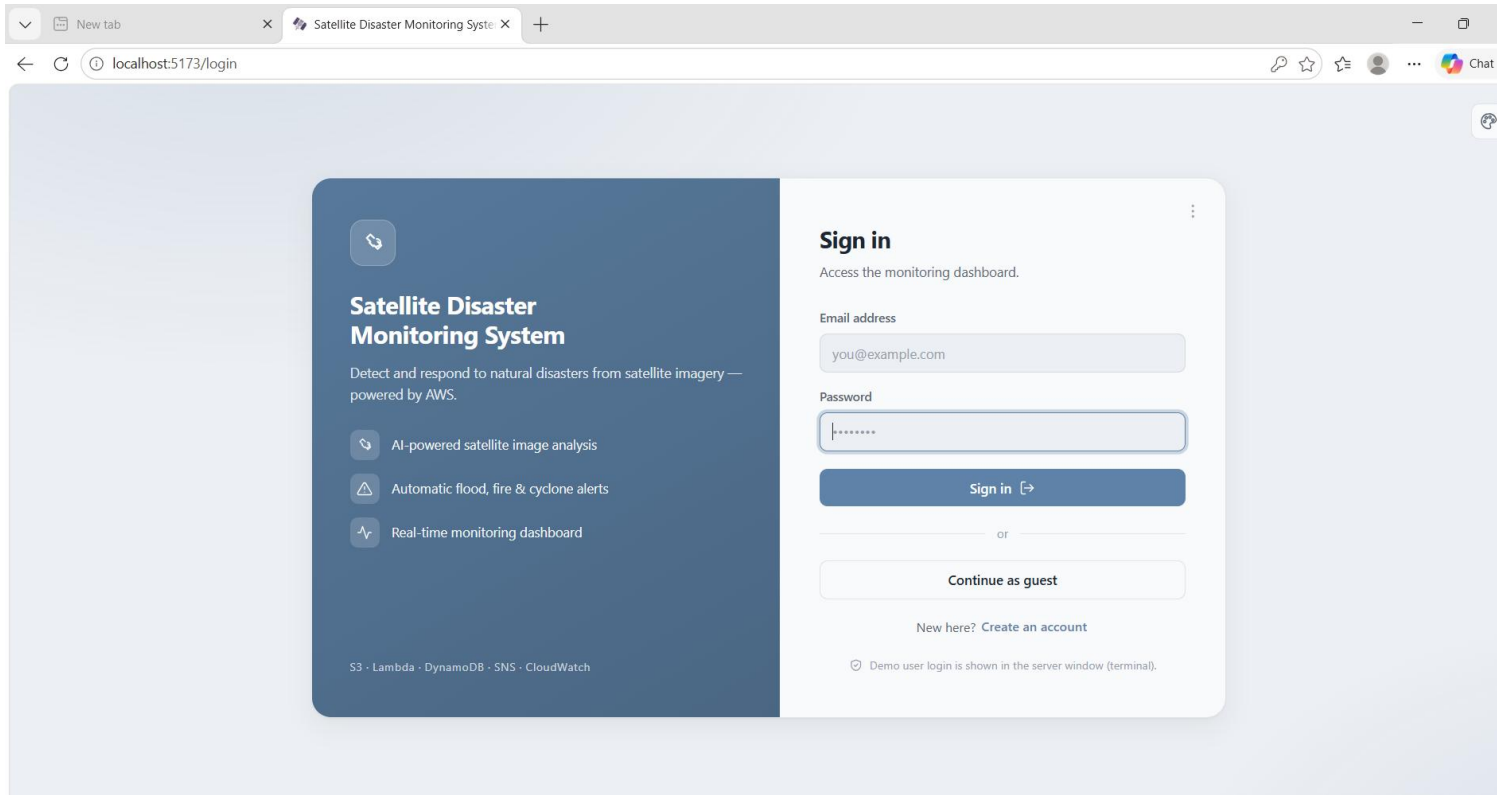
Stores execution logs.

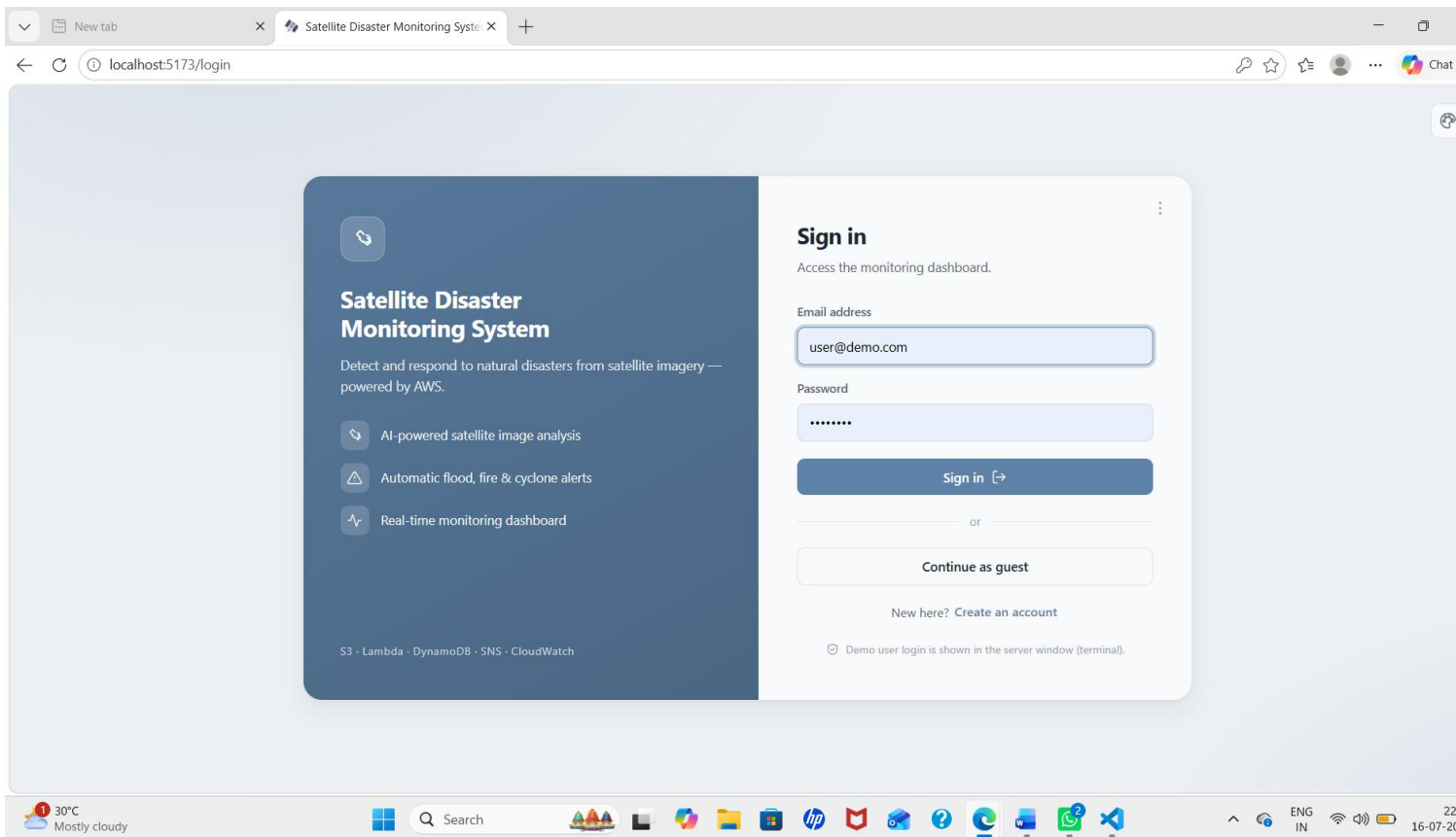
Helps identify and troubleshoot errors.



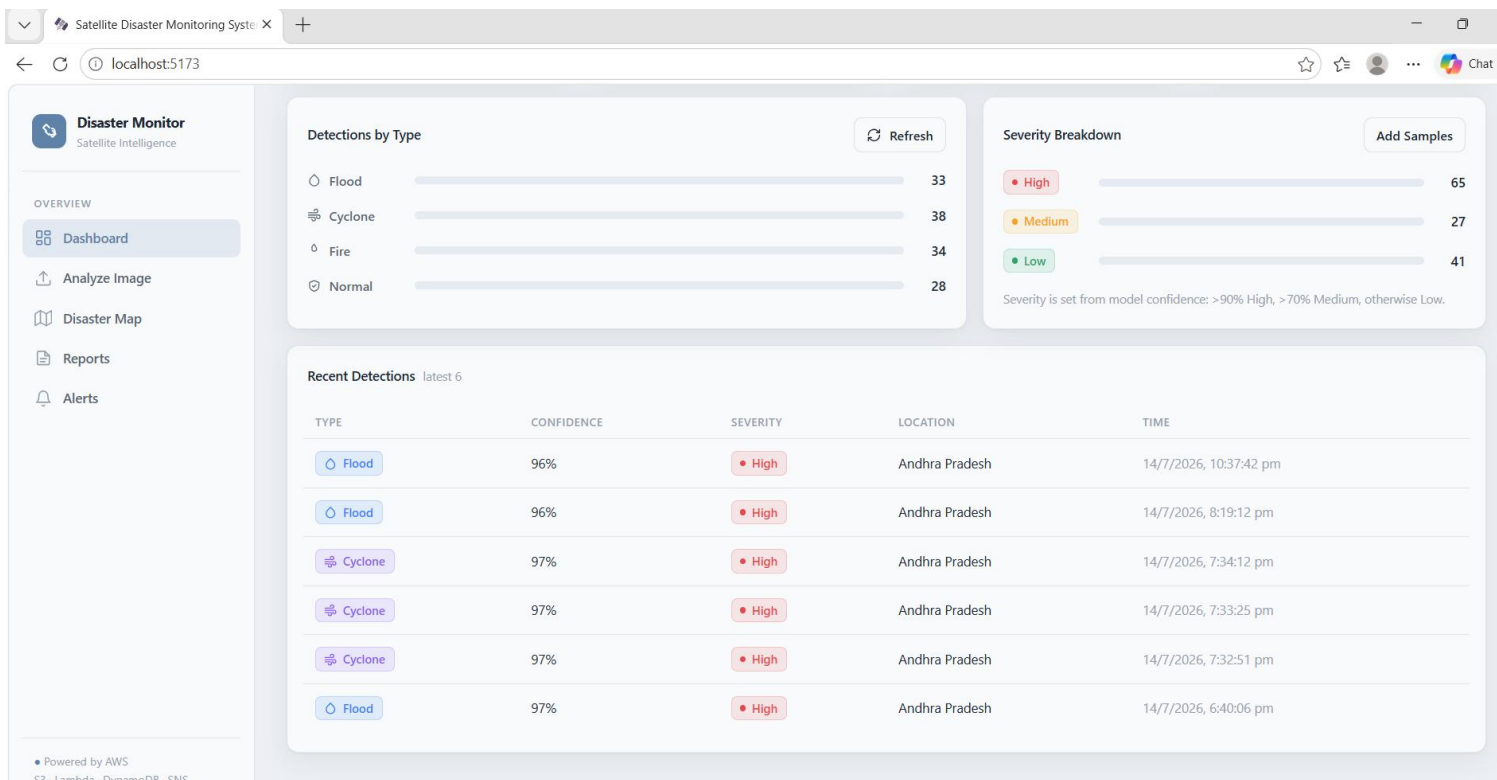
8. Project Output

8.1: Login Page





8.2: Dashboard



**Disaster Monitor**
Satellite Intelligence

OVERVIEW

- Dashboard
- Analyze Image
- Disaster Map
- Reports
- Alerts

• Powered by AWS
S3 · Lambda · DynamoDB · SNS

Analyze Image

Upload a satellite image for disaster classification

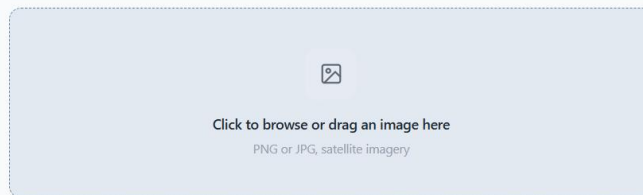
Live · AWS



DU

Demo User

Input



Location

Andhra Pradesh

Run Analysis

Tip: name a file `Flood_1.jpg` to force that class for a demo.

Result



Awaiting analysis

Run analysis to see the predicted class, confidence, and severity.

33°C
Cloudy



ENG IN 15-07-20

**Disaster Monitor**
Satellite Intelligence

OVERVIEW

- Dashboard
- Analyze Image
- Disaster Map
- Reports
- Alerts

• Powered by AWS
S3 · Lambda · DynamoDB · SNS

Disaster Map

Interactive map of detected incidents across India

Live · AWS

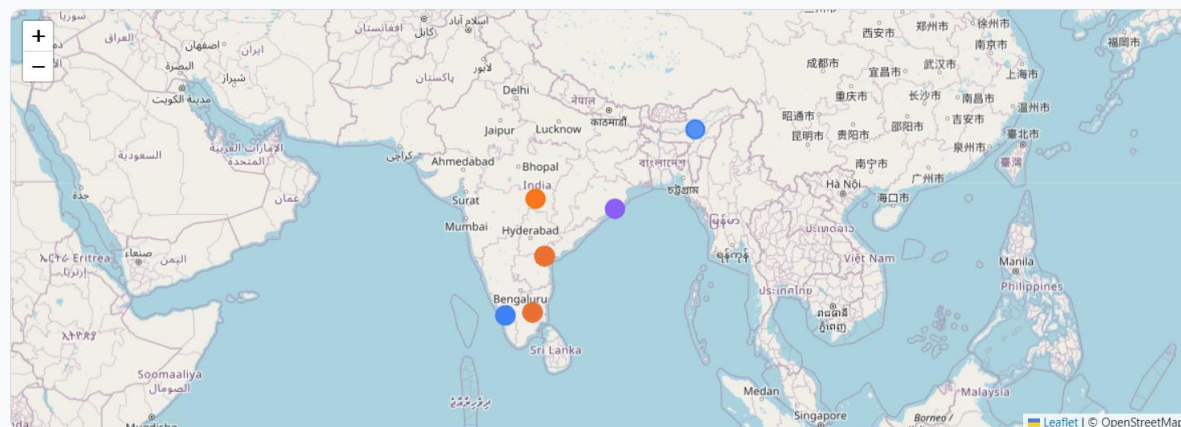


DU

Demo User

Flood Fire Cyclone

Refresh



Click a marker for details. Open full reports →

33°C
Cloudy



ENG IN 15-07-20

Satellite Disaster Monitoring System

localhost:5173/reports

Disaster Monitor

Satellite Intelligence

OVERVIEW

Dashboard

Analyze Image

Disaster Map

Reports

Alerts

Powered by AWS

S3 · Lambda · DynamoDB · SNS

Reports

133 records

Live · AWS

DU Demo User

Search by location

All types

All severity

CSV

TYPE	CONFIDENCE	SEVERITY	LOCATION	STATUS	TIME	
Flood	96%	High	Andhra Pradesh	Active	14/7/2026, 10:37:42 pm	View →
Flood	96%	High	Andhra Pradesh	Active	14/7/2026, 8:19:12 pm	View →
Cyclone	97%	High	Andhra Pradesh	Active	14/7/2026, 7:34:12 pm	View →
Cyclone	97%	High	Andhra Pradesh	Active	14/7/2026, 7:33:25 pm	View →
Cyclone	97%	High	Andhra Pradesh	Active	14/7/2026, 7:32:51 pm	View →
Flood	97%	High	Andhra Pradesh	Active	14/7/2026, 6:40:06 pm	View →
Cyclone	97%	High	Andhra Pradesh	Active	14/7/2026, 6:03:32 pm	View →
Flood	97%	High	Andhra Pradesh	Active	14/7/2026, 5:40:24 pm	View →
Flood	96%	High	Andhra Pradesh	Active	14/7/2026, 5:39:41 pm	View →
Cyclone	97%	High	Andhra Pradesh	Active	14/7/2026, 4:19:58 pm	View →

33°C

Cloudy

Search

15-07-2026

Satellite Disaster Monitoring System

localhost:5173/alerts

Disaster Monitor

Satellite Intelligence

OVERVIEW

Dashboard

Analyze Image

Disaster Map

Reports

Alerts

Powered by AWS

S3 · Lambda · DynamoDB · SNS

Alerts

High-severity detections that triggered SNS notifications

Live · AWS

DU Demo User

Flood

Andhra Pradesh

Confidence - 96%

14/7/2026, 10:37:42 pm

Flood

Andhra Pradesh

Confidence - 96%

14/7/2026, 8:19:12 pm

Cyclone

Andhra Pradesh

Confidence - 97%

14/7/2026, 7:34:12 pm

Cyclone

Andhra Pradesh

Confidence - 97%

14/7/2026, 7:33:25 pm

Cyclone

Andhra Pradesh

Confidence - 97%

14/7/2026, 7:32:51 pm

Flood

Andhra Pradesh

Confidence - 97%

14/7/2026, 6:40:06 pm

Cyclone

Andhra Pradesh

Confidence - 97%

Flood

Andhra Pradesh

Confidence - 97%

Flood

Andhra Pradesh

Confidence - 96%

33°C

Cloudy

Search

15-07-2026

8.3: Upload Satellite Image

Satellite Disaster Monitoring

S3 buckets | S3 | eu-north-1

IAM users | IAM | Global

List tables | Amazon DynamoDB

API Gateway

localhost:5173/upload

 **Disaster Monitor**
Satellite Intelligence

OVERVIEW

Dashboard

Analyze Image

Disaster Map

Reports

Alerts

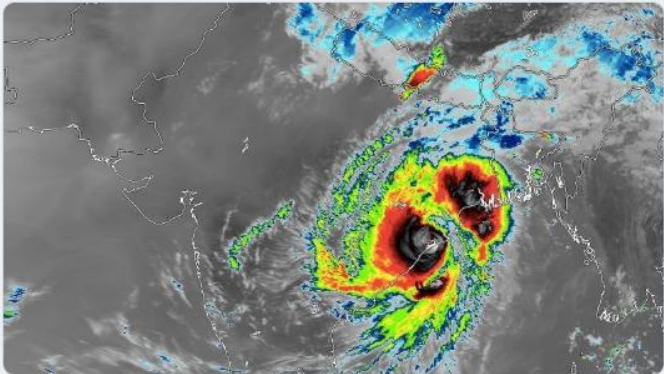
Powered by AWS

S3 · Lambda · DynamoDB · SNS

Analyze Image

Upload a satellite image for disaster classification

Input



Location

Andhra Pradesh

Run Analysis

Clear

Tip: name a file `flood_1.jpg` to force that class for a demo.

8.4: AI Prediction
Result

Disaster Monitor
Satellite Intelligence

OVERVIEW

Dashboard

Analyze Image

Disaster Map

Reports

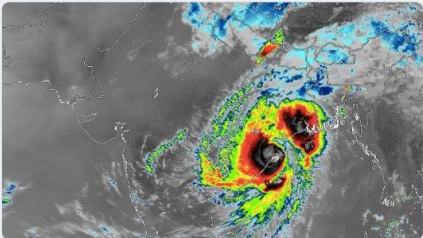
Alerts

Powered by AWS
S3 · Lambda · DynamoDB · SNS

Analyze Image

Upload a satellite image for disaster classification

Input



Location
Andhra Pradesh

Run Analysis

Clear

Tip: name a file `Flood_1.jpg` to force that class for a demo.

Result

Cyclone

Confidence · 96%

Severity

High

Location

Andhra Pradesh

Lifecycle

Active

Status

Alert dispatched (SNS)

Storage

s3://amazonbucket1332/c2cc41146...

Time

15/7/2026, 8:26:42 pm

High severity. An alert was dispatched via SNS to the authorities

View details

Open map

8.5: Disaster Reports

Disaster Monitor
Satellite Intelligence

OVERVIEW

Dashboard

Analyze Image

Disaster Map

Reports

Alerts

Powered by AWS
S3 · Lambda · DynamoDB · SNS

Reports

133 records

Search by location

All types

All severity

Refresh

Download CSV

TYPE	CONFIDENCE	SEVERITY	LOCATION	STATUS	TIME	
Flood	96%	High	Andhra Pradesh	Active	14/7/2026, 10:37:42 pm	View →
Flood	96%	High	Andhra Pradesh	Active	14/7/2026, 8:19:12 pm	View →
Cyclone	97%	High	Andhra Pradesh	Active	14/7/2026, 7:34:12 pm	View →
Cyclone	97%	High	Andhra Pradesh	Active	14/7/2026, 7:33:25 pm	View →
Cyclone	97%	High	Andhra Pradesh	Active	14/7/2026, 7:32:51 pm	View →
Flood	97%	High	Andhra Pradesh	Active	14/7/2026, 6:40:06 pm	View →
Cyclone	97%	High	Andhra Pradesh	Active	14/7/2026, 6:03:32 pm	View →
Flood	97%	High	Andhra Pradesh	Active	14/7/2026, 5:40:24 pm	View →
Flood	96%	High	Andhra Pradesh	Active	14/7/2026, 5:39:41 pm	View →
Cyclone	97%	High	Andhra Pradesh	Active	14/7/2026, 4:19:58 pm	View →

8.6: Interactive Map

localhost:5173/map

Chat

Disaster Monitor

Satellite Intelligence

OVERVIEW

Dashboard

Analyze Image

Disaster Map

Reports

Alerts

Disaster Map

Interactive map of detected incidents across India

Live - AWS

DU Demo User

FloodFireCyclone

Refresh

Map of India and surrounding regions showing detected incidents. Legend: Flood (Blue), Fire (Orange), Cyclone (Purple). Incidents are marked on the map. Cities labeled include Delhi, Jaipur, Lucknow, Ahmedabad, Bhopal, Mumbai, Surat, Hyderabad, Bengaluru, Sri Lanka, and various cities in China, Vietnam, Philippines, and Malaysia.

Click a marker for details. Open full reports →

Powered by AWS

S3 - Lambda - DynamoDB - SNS

localhost:5173/map

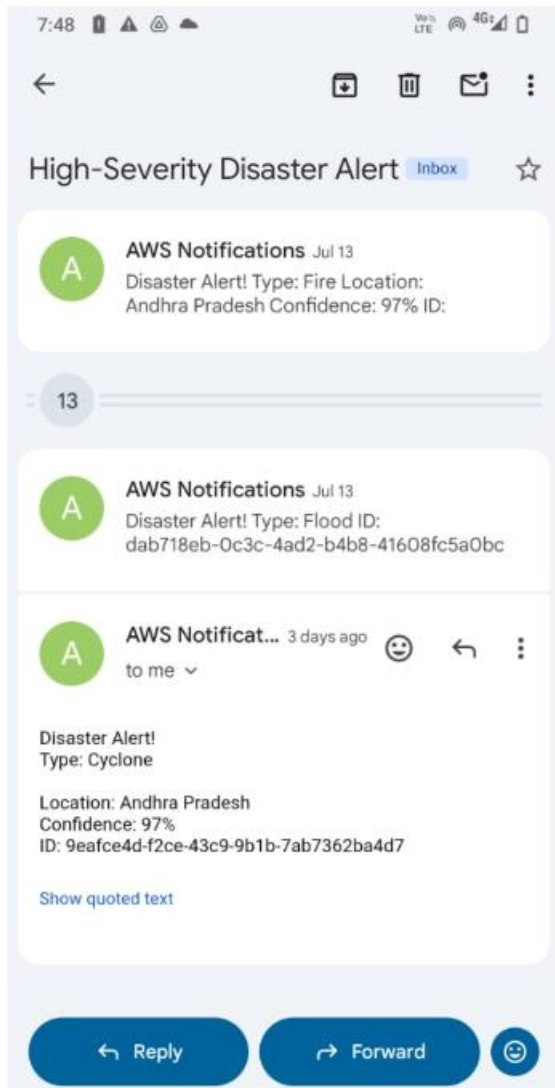
33°C Cloudy

Search

ENG IN

15-07-20

8.7: SNS Email Alert



8.8: user upload image stored in Amazon S3 Bucket

← → ↻ 🔒 https://eu-north-1.console.aws.amazon.com/s3/buckets/1amazonbucket1332?region=eu-north-1&tab=objects

aws [Alt+S] Europe (Stockholm) project (2407972123)

Amazon S3 > Buckets > 1amazonbucket1332

Amazon S3

- ▼ Buckets
 - General purpose buckets
 - Directory buckets
 - Table buckets
 - Vector buckets
- ▼ Files
 - File systems
- ▼ Access management and security
 - Access Points
 - Access Points for FSx
 - Access Grants
 - IAM Access Analyzer
- ▼ Storage management and insights
 - Storage Lens
 - Batch Operations

Objects (19)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix Show versions

☐	Name	Type	Last modified	Size	Storage class
☐	954e80a46ef24944947d828e5d1ca3fd.jpg	jpg	July 13, 2026, 04:51:48 (UTC+05:30)	65.4 KB	Standard
☐	c3d1babf6bfe4420a4ffa50f32dfaefb.jpg	jpg	July 13, 2026, 09:03:55 (UTC+05:30)	39.4 KB	Standard
☐	c82357e7e92d46f1bf92f3b7bf895e80.jpg	jpg	July 13, 2026, 07:43:56 (UTC+05:30)	11.4 KB	Standard
☐	d7e84104cab24ebcb30f1e5d24221b36.jpg	jpg	July 13, 2026, 04:52:48 (UTC+05:30)	66.5 KB	Standard
☐	fae871fbd79441e09b7d6b9d3d4a5725.jpg	jpg	July 13, 2026, 10:09:29 (UTC+05:30)	4.6 KB	Standard
☐	flood.jpg	jpg	July 14, 2026, 19:44:05 (UTC+05:30)	82.1 KB	Standard

8.9: user upload image stored in DynamoDB Table

← ↻ https://eu-north-1.console.aws.amazon.com/dynamodbv2/home?region=eu-north-1#item-explorer?maximize=true&table=disateralert

aws [Alt+S] Ask Amazon Q Europe (Stockholm) project (240797212332) project

DynamoDB > Explore items > disateralert

DynamoDB

- Dashboard
- Tables
- Explore items
- PartiQL editor
- Backups
- Exports to S3
- Imports from S3
- Integrations
- Reserved capacity
- Settings

▼ DAX

- Clusters
- Subnet groups
- Parameter groups
- Events

Table: disateralert - Items returned (27)

Scan started on July 15, 2026, 14:01:36

☐	report-id (String)	bucket	confidence	disaster_type	disaster-type
☐	6627f1cb-192b-4872-...		96	Flood	
☐	3bba87c6fb19499b9b...	1amazonbu...			
☐	486b4ad8aed447cfba...	1amazonbu...			
☐	images/fire.jpg	1amazonbu...			
☐	384d141385b3403a9...	1amazonbu...			
☐	2c76314d41d44513b...	1amazonbu...			
☐	c82357e92d46f1bf...	1amazonbu...			
☐	91d8a3c1-db96-414e-...		87	Flood	
☐	R003				flood
☐	696e46d0b91d47d29...	1amazonbu...			
☐	87aae944e18c48d1b4...	1amazonbu...			

CloudShell Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

34°C Mostly cloudy Search

8.10: Lambda Function

aws [Alt+S] Search

Lambda > Functions

Lambda

- Dashboard
- Applications

Functions

▼ Function resources

- Capacity providers
- Code signing configurations
- Event source mappings
- Layers
- Replicas

Documentation ↗

Serverless workshops ↗

▼ Related AWS resources

- Step Functions state machines

Search functions by name 🔍

Functions (1)

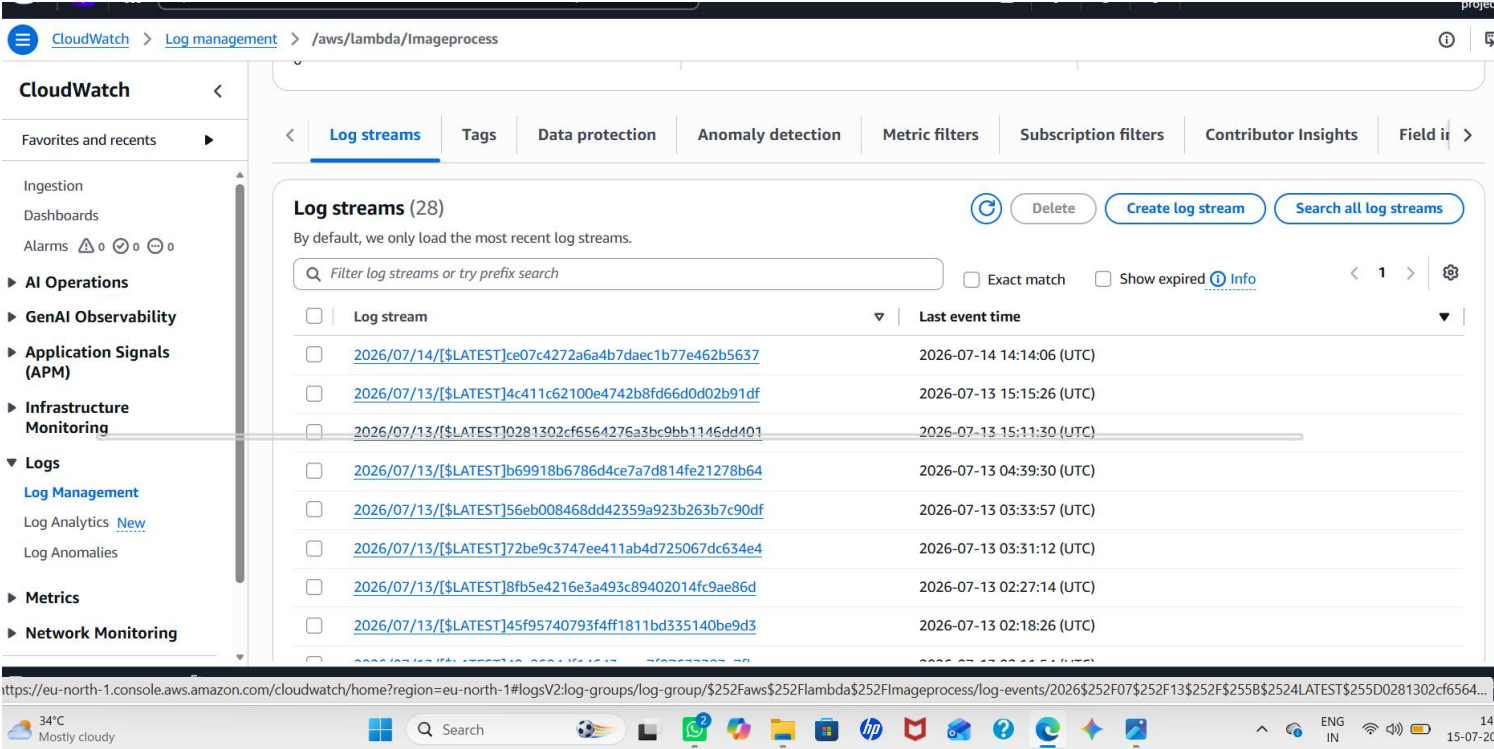
Last fetched 15/7/2026, 1:56:03 pm

Functions table allows you to query function details. Use Ctrl + K to start a in-console resource search.

Search by attributes or search by keyword

☐	Function name	Description	Package type	Runtime	Type	Last modified
☐	Imageprocess	-	Zip	Python 3.13	Standard	3 days ago

8.11: CloudWatch Logs



9 Project Links

Resource	URL
GitHub Repository	https://github.com/podruvenkatadevi/Satellite-based-disaster-monitoring-and-alert-system
Local Frontend (Dev)	http://localhost:5173
Local Backend API	http://127.0.0.1:8000
API Health Check	http://127.0.0.1:8000/api/health
API	http://127.0.0.1:8000/docs

Demo Logins:		
Role	Email	Password
User	user@demo.com	User@123
Admin	admin@demo.com	Admin@123

10. Conclusion

The Satellite-Based Disaster Monitoring System provides an intelligent solution for detecting and monitoring natural disasters through AI-powered satellite image analysis. By classifying disasters automatically, assigning severity levels, visualizing impact on an interactive map, and sending real-time alerts, the system helps disaster management teams respond faster and more effectively.

With modular FastAPI backend, React frontend, multi-tier AI fallback, and AWS cloud integration, the platform offers a reliable and scalable solution for educational institutions, government agencies, and disaster response teams.

11. References

1. React Documentation. React – The library for web and native user interfaces. <https://react.dev/>
2. Vite Documentation. Next Generation Frontend Tooling. <https://vitejs.dev/>
3. FastAPI Documentation. FastAPI framework for building APIs. <https://fastapi.tiangolo.com/>
4. Python Software Foundation. Python Documentation. <https://docs.python.org/3/>
5. PyTorch Documentation. An open source machine learning framework. <https://pytorch.org/docs/>
6. Hugging Face Transformers. CLIP Model Documentation. <https://huggingface.co/docs/transformers/>
7. Leaflet Documentation. An open-source JavaScript library for mobile-friendly interactive maps. <https://leafletjs.com/>
8. AWS Documentation. Amazon S3, DynamoDB, SNS, and Lambda. <https://docs.aws.amazon.com/>
9. Uvicorn Documentation. The lightning-fast ASGI server. <https://www.uvicorn.org/>

THANKYOU